

Natural Language Processing

Tel Aviv University

**Assignment 2: Language Models & Transformers**Due Date: *June 16, 2026*

Lecturer: Dr. Mor Pipek, TA: Idan Tarshish

## Preliminaries

**Submission Instructions** This assignment includes both theoretical and coding questions. Your submission should be one zip file that includes:

1. A single PDF file in digital format (we recommend using Overleaf, but you may use any digital editor you prefer). The PDF should contain:
  - (a) Answers to Q1a+b+d.
  - (b) Answer to Q2a and the plot from Q2c
  - (c) Answers to all three sub questions of Q3.
  - (d) Answers to Q4a+b.
  - (e) Answers to all sub questions of Q5.
  - (f) AI disclosure & Reflection (Q6)
2. The directory q1\_2\_3, including all its files, with the **completed** .py files, the saved\_params\_40000.npy and the **completed** q2\_char\_rnn\_generation
3. The **completed** notebook q4\_transformer.

Submit your solution via Moodle. Your submission should be a single zip file named <id1>\_<id2>\_<id3>.zip, where id1 is the ID of the first student. The zip file should include all of the above components.

## Notes

- Only one student needs to submit.
- Your implementation should be efficient and vectorized whenever possible (i.e., use numpy matrix operations rather than for loops). A non-vectorized implementation will not receive full credit!
- Your code should follow coding standards (well-documented, self-explained, meaningful naming, PEP8).

## 1 Word-Level Neural Bi-gram Language Model (20 pt)

In this question, you will implement and train neural language model, and evaluate it on using perplexity.

- (a) Derive the gradient with respect to the input of a softmax function when cross entropy loss is used for evaluation, i.e., find the gradients with respect to the softmax input vector  $\theta$ , when the prediction is made by  $\hat{\mathbf{y}} = \text{softmax}(\theta)$ . Cross entropy and softmax are defined as:

$$\text{CE}(\mathbf{y}, \hat{\mathbf{y}}) = - \sum_i y_i \cdot \log(\hat{y}_i)$$

$$\text{softmax}(\theta)_i = \frac{\exp(\theta_i)}{\sum_j \exp(\theta_j)}$$

The gold vector  $\mathbf{y}$  is a one-hot vector, and the predicted vector  $\hat{\mathbf{y}}$  is a probability distribution over the output space.

- (b) Derive the gradients with respect to the input  $\mathbf{x}$  in a one-hidden-layer neural network (i.e., find  $\frac{\partial J}{\partial \mathbf{x}}$ , where  $J$  is the cross entropy loss  $\text{CE}(\mathbf{y}, \hat{\mathbf{y}})$ ). The neural network employs a sigmoid activation function for the hidden layer, and a softmax for the output layer. Assume a one-hot label vector  $\mathbf{y}$  is used. The network is defined as:

$$\mathbf{h} = \sigma(\mathbf{x}\mathbf{W}_1 + \mathbf{b}_1),$$

$$\hat{\mathbf{y}} = \text{softmax}(\mathbf{h}\mathbf{W}_2 + \mathbf{b}_2).$$

The dimensions of the vectors and matrices are  $\mathbf{x} \in \mathbb{R}^{1 \times D_x}$ ,  $\mathbf{h} \in \mathbb{R}^{1 \times D_h}$ ,  $\hat{\mathbf{y}} \in \mathbb{R}^{1 \times D_y}$ ,  $\mathbf{y} \in \mathbb{R}^{1 \times D_y}$ . The dimensions of the parameters are  $\mathbf{W}_1 \in \mathbb{R}^{D_x \times D_h}$ ,  $\mathbf{W}_2 \in \mathbb{R}^{D_h \times D_y}$ ,  $\mathbf{b}_1 \in \mathbb{R}^{1 \times D_h}$ ,  $\mathbf{b}_2 \in \mathbb{R}^{1 \times D_y}$ .

- (c) Implement the forward and backward passes for a neural network with one sigmoid hidden layer. Fill in your implementation in `q1c_neural.py`. Sanity check your implementation with `python q1c_neural.py`.
- (d) GloVe ([Global Vectors](#)) embeddings are a type of word embeddings that represent words as vectors in a high-dimensional space, based on the co-occurrence statistics of words in a corpus. They are related to the skip-gram embeddings you saw in class in that they both aim to capture the semantic and syntactic relationships between words, but GloVe embeddings incorporate global corpus-level information in addition to local context information. In this section you will be using GloVe embeddings to represent the vocabulary.

Use the neural network to implement a bigram language model in `q1d_neural_lm.py`. Use GloVe embeddings to represent the vocabulary (`data/lm/vocab_embeddings.glove.txt`). Implement the `lm_wrapper` function, that is used by `sgd` to sample the gradient, and the `eval_neural_lm` function that is used for model evaluation. **Report the dev perplexity in your written solution.** Don't forget to save `saved_params_40000.npy` and include it in your submission zip!

## 2 Generating Shakespeare with a Char-level LM (15 pt)

In the previous section we dealt with word-level language models. But looking again at section 1, there is nothing that constraints us to using words as the basic elements in our model. The model we analyzed in section 1 could just as well be character-based - just replace "word" with "character", and you are good to go. In this section we will train a small character-based language model that will help us generate Shakespearean-like (emphasis on the like...) texts.

- (a) Can you think of an advantage a character-based language model could have over a word-based language model? And what about the other way around: can you think of an advantage a word-based language model could have over a character-based language model? **Answer in 3-6 sentences.**

- (b) Follow the instructions and complete the code in the attached notebook:

`q2_char_rnn_generation.ipynb`.

- (c) Plotting the losses that were computed during training can provide a further indication that the network was indeed learning. Add your plot to the written solution.

### 3 Perplexity (20 pt)

- (a) Show that perplexity calculated using the natural logarithm  $\ln(x)$  is equal to perplexity calculated using  $\log_2(x)$ . i.e:

$$2^{-\frac{1}{M} \sum_{i=1}^M \log_2 p(s_i | s_1, \dots, s_{i-1})} = e^{-\frac{1}{M} \sum_{i=1}^M \ln p(s_i | s_1, \dots, s_{i-1})}$$

- (b) In this section you will be computing the perplexity of your previous trained models on two different passages - Wikipedia text and Shakespearean Text. You may and should add the code for doing so to the code you wrote in the previous sections - You can use the `saved_params` files and the `load_saved_params` modalities in Q1 (You don't need to train a new model). You can add cells to Q2's notebook as well.

All the data you need is in the assignment .zip. `shakespeare_for_perplexity.txt` contains a subset from the Shakespeare dataset, and `wikipedia_for_perplexity.txt` contains a certain passage from Wikipedia. For your convenience, both passages are also provided in a POS-tagged format, similar to the datasets used for training and testing in Section 1.

Please compute the perplexity of the bi-gram LM (from Section 1) and the character-level LM (from Section 2) on both these passages. **Note: To receive full credit for this question, you must include at least one code block that calculates Perplexity. This can be located in either the Q2 notebook or the Q1 files.** Report your results in a table of this format:

| Model-Data                            | Wikipedia Text | Shakespearean Text |
|---------------------------------------|----------------|--------------------|
| Word-Level Neural Bi-gram (Section 1) | XX             | XX                 |
| Character-level RNN (Section 2)       | XX             | XX                 |

- (c) Try to explain the results you have got. Particularly, why there might be large gaps in perplexity, while looking at different passages. **Answer in 3-6 sentences.**

### 4 Implementing a Transformer Model (20 pt)

In this question we will implement a full functioning transformer, using toy data and building the architecture from the building blocks. **Open the attached notebook (`q4_transformer.ipynb`) and fill in the missing code parts according to the instructions.**

After completing and running the notebook, answer these theoretical questions in your submitted pdf file:

- (a) Observe the assertion of

`assert d_model % num_heads == 0`

at the beginning of the ‘`__init__`’ function in the Multiple Head Attention Class (Part 1 of the notebook) . Why is this assertion needed?

- (b) As mentioned throughout the notebook, our Transformer implementation is autoregressive. What changes would be required to adapt it into a Masked Language Model (MLM), such as BERT?
- (i) Which components should be added or removed from the architecture?
- (ii) How would the masking strategy change, both for attention (e.g., no-peek masking) and for token masking?

## 5 Attention Exploration (25 pt)

Multi-head self-attention is the core modeling component of Transformers. In this question, we'll get some practice working with the self-attention equations, and motivate why multi-headed self-attention can be preferable to single-headed self-attention.

Recall that attention can be viewed as an operation on a *query* vector  $q \in \mathbb{R}^d$ , a set of *value* vectors  $\{v_1, \dots, v_n\}, v_i \in \mathbb{R}^d$ , and a set of *key* vectors  $\{k_1, \dots, k_n\}, k_i \in \mathbb{R}^d$ , specified as follows:

$$c = \sum_{i=1}^n v_i \alpha_i \quad (1)$$

$$\alpha_i = \frac{\exp(k_i^\top q)}{\sum_{j=1}^n \exp(k_j^\top q)} \quad (2)$$

with  $\alpha = \{\alpha_1, \dots, \alpha_n\}$  termed the “attention weights”. Observe that the output  $c \in \mathbb{R}^d$  is an average over the value vectors weighted with respect to  $\alpha$ .

1. **Copying in attention.** One advantage of attention is that it's particularly easy to “copy” a value vector to the output  $c$ . In this problem, we'll motivate why this is the case.

- (a) **Explain** why  $\alpha$  can be interpreted as a categorical probability distribution.
- (b) The distribution  $\alpha$  is typically relatively “diffuse”; the probability mass is spread out between many different  $\alpha_i$ . However, this is not always the case. **Describe** (in one sentence) under what conditions the categorical distribution  $\alpha$  puts almost all of its weight on some  $\alpha_j$ , where  $j \in \{1, \dots, n\}$  (i.e.  $\alpha_j \gg \sum_{i \neq j} \alpha_i$ ). What must be true about the query  $q$  and/or the keys  $\{k_1, \dots, k_n\}$ ?
- (c) Under the conditions you gave in (b), **describe** the output  $c$ .
- (d) **Explain** (in two sentences or fewer) what your answers to (b) and (c) mean together, intuitively.

2. **An average of two.** Instead of focusing on just one vector  $v_j$ , a Transformer model might want to incorporate information from *multiple* source vectors. Consider the case where we instead want to incorporate information from **two** vectors  $v_a$  and  $v_b$ , with corresponding key vectors  $k_a$  and  $k_b$ .

- (a) How should we combine two  $d$ -dimensional vectors  $v_a, v_b$  into one output vector  $c$  in a way that preserves information from both vectors? In machine learning, one common way to do so is to take the average:  $c = \frac{1}{2}(v_a + v_b)$ . It might seem hard to extract information about the original vectors  $v_a$  and  $v_b$  from the resulting  $c$ , but under certain conditions one can do so. In

this problem, we'll see why this is the case.

Suppose that although we don't know  $v_a$  or  $v_b$ , we do know that  $v_a$  lies in a subspace  $A$  formed by the  $m$  basis vectors  $\{a_1, a_2, \dots, a_m\}$ , while  $v_b$  lies in a subspace  $B$  formed by the  $p$  basis vectors  $\{b_1, b_2, \dots, b_p\}$ . (This means that any  $v_a$  can be expressed as a linear combination of its basis vectors, as can  $v_b$ . All basis vectors have norm 1 and are orthogonal to each other.) Additionally, suppose that the two subspaces are orthogonal; i.e.  $a_j^\top b_k = 0$  for all  $j, k$ .

Using the basis vectors  $\{a_1, a_2, \dots, a_m\}$ , construct a matrix  $M$  such that for arbitrary vectors  $v_a \in A$  and  $v_b \in B$ , we can use  $M$  to extract  $v_a$  from the sum vector  $s = v_a + v_b$ . In other words, we want to construct  $M$  such that for any  $v_a, v_b$ ,  $Ms = v_a$ . Show that  $Ms = v_a$  holds for your  $M$ .

**Hint:** First, come up with a rank-1 matrix  $A_i$  such that  $A_i a_i = a_i$  and  $A_i b = 0$  for every  $b$  such that  $b^\top a_i = 0$ . Now, use those  $A_i$ s to construct the desired matrix  $M$ .

- (b) As before, let  $v_a$  and  $v_b$  be two value vectors corresponding to key vectors  $k_a$  and  $k_b$ , respectively. Assume that (1) all key vectors are orthogonal, so  $k_i^\top k_j = 0$  for all  $i \neq j$ ; and (2) all key vectors have norm 1.<sup>1</sup> **Find an expression** for a query vector  $q$  such that  $c \approx \frac{1}{2}(v_a + v_b)$ , and justify your answer.<sup>2</sup>

**3. Drawbacks of single-headed attention:** In the previous part, we saw how it was *possible* for a single-headed attention to focus equally on two values. The same concept could easily be extended to any subset of values. In this question we'll see why it's not a *practical* solution. Consider a set of key vectors  $\{k_1, \dots, k_n\}$  that are now randomly sampled,  $k_i \sim \mathcal{N}(\mu_i, \Sigma_i)$ , where the means  $\mu_i \in \mathbb{R}^d$  are known to you, but the covariances  $\Sigma_i$  are unknown. Further, assume that the means  $\mu_i$  are all perpendicular;  $\mu_i^\top \mu_j = 0$  if  $i \neq j$ , and unit norm,  $\|\mu_i\| = 1$ .

- (a) Assume that the covariance matrices are  $\Sigma_i = \alpha I, \forall i \in \{1, 2, \dots, n\}$ , for vanishingly small  $\alpha$ . Design a query  $q$  in terms of the  $\mu_i$  such that as before,  $c \approx \frac{1}{2}(v_a + v_b)$ , and provide a brief argument as to why it works.
- (b) Though single-headed attention is resistant to small perturbations in the keys, some types of larger perturbations may pose a bigger issue. Specifically, in some cases, one key vector  $k_a$  may be larger or smaller in norm than the others, while still pointing in the same direction as  $\mu_a$ . As an example, let us consider a covariance for item  $a$  as  $\Sigma_a = \alpha I + \frac{1}{2}(\mu_a \mu_a^\top)$  for vanishingly small  $\alpha$  (as shown in figure 1). This causes  $k_a$  to point in roughly the same direction as  $\mu_a$ , but with large variances in magnitude. Further, let  $\Sigma_i = \alpha I$  for all  $i \neq a$ .

When you sample  $\{k_1, \dots, k_n\}$  multiple times, and use the  $q$  vector that you defined in part (a), what do you expect the vector  $c$  will look like qualitatively for different samples? Think about how it differs from part (a) and how  $c$ 's variance would be affected.

**4. Benefits of multi-headed attention:** Now we'll see some of the power of multi-headed attention. We'll consider a simple version of multi-headed attention which is identical to single-headed self-attention as we've presented it in this homework, except two query vectors ( $q_1$  and  $q_2$ ) are defined, which leads to a pair of vectors ( $c_1$  and  $c_2$ ), each the output of single-headed attention given its respective query vector. The final output of the multi-headed attention is their average,  $\frac{1}{2}(c_1 + c_2)$ . As in question 1(3), consider a set of key vectors  $\{k_1, \dots, k_n\}$  that are randomly sampled,  $k_i \sim \mathcal{N}(\mu_i, \Sigma_i)$ , where the means  $\mu_i$  are known to you, but the covariances  $\Sigma_i$  are unknown. Also as

<sup>1</sup>Recall that a vector  $x$  has norm 1 iff  $x^\top x = 1$ .

<sup>2</sup>Hint: while the softmax function will never *exactly* average the two vectors, you can get close by using a large scalar multiple in the expression.

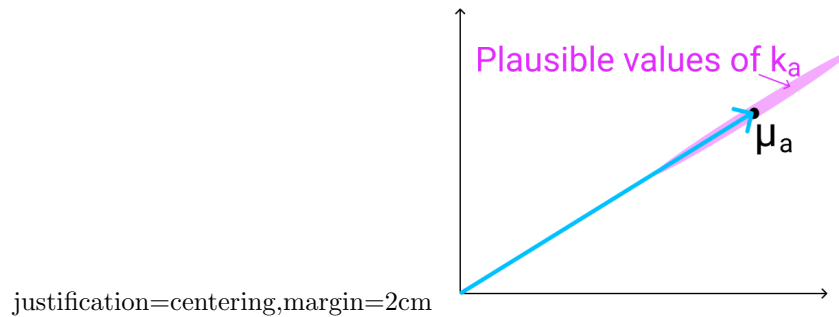


Figure 1: The vector  $\mu_a$  (shown here in 2D as an example), with the range of possible values of  $k_a$  shown in red. As mentioned previously,  $k_a$  points in roughly the same direction as  $\mu_a$ , but may have larger or smaller magnitude.

before, assume that the means  $\mu_i$  are mutually orthogonal;  $\mu_i^\top \mu_j = 0$  if  $i \neq j$ , and unit norm,  $\|\mu_i\| = 1$ .

- (a) Assume that the covariance matrices are  $\Sigma_i = \alpha I$ , for vanishingly small  $\alpha$ . Design  $q_1$  and  $q_2$  such that  $c$  is approximately equal to  $\frac{1}{2}(v_a + v_b)$ . Note that  $q_1$  and  $q_2$  should have different expressions.
- (b) Assume that the covariance matrices are  $\Sigma_a = \alpha I + \frac{1}{2}(\mu_a \mu_a^\top)$  for vanishingly small  $\alpha$ , and  $\Sigma_i = \alpha I$  for all  $i \neq a$ . Take the query vectors  $q_1$  and  $q_2$  that you designed in part i. What, qualitatively, do you expect the output  $c$  to look like across different samples of the key vectors? Explain briefly in terms of variance in  $c_1$  and  $c_2$ . You can ignore cases in which  $k_a^\top q_i < 0$ .

**Acknowledgements** This question was adapted from Stanford’s CS224n course. Their contributions are greatly appreciated.

## 6 AI Disclosure & Reflection

This question is mandatory, but it will not be graded. Furthermore, its content does not impact the grading and evaluation of the rest of your submission. The goal is to promote transparency and responsible LLM and AI usage.

Answer the following parts briefly (1 paragraph):

- a) Did you use AI in the work on this assignment? If yes, please name the specific tools/models used (e.g., ChatGPT—model name, Copilot, Claude, etc.).
- b) How and why did you use it – which parts of the work and for what purpose (e.g., brainstorming, outlining, code generation, debugging, editing, evaluation).
- c) Reflection: What helped or hindered? What is your takeaway from this experience?

If you did not use AI, write: “No AI used”, and add a sentence explaining why.